Universidad Peruana Cayetano Heredia
Facultad de Ciencias e Ingeniería
Departamento de Ciencias Exactas

Programación Avanzada 2025 (IB)

Lab. 10.2. Listas Enlazadas. Pilas (Stacks)

Octubre 24, 2025

Cree una carpeta (folder) en el disco D, nómbrela con su apellido paterno seguido de su código. Ejemplo: LOPEZ12345

Resuelva los siguientes ejercicios usando listas enlazadas.

1. Descargue la clase `LinkedList` completa. Genere una lista enlazada de ejemplo y pruebe el funcionamiento del iterador.

2. Escriba un programa que solicite el ingreso de dos números enteros positivos y determine la suma de ambos números usando listas enlazadas.

   a) Para cada número genere una lista enlazada. Cada dígito de un número se almacena en un nodo de una lista enlazada.

   b) Sumar elemento a elemento los nodos de las listas enlazadas que representan a los números generando al mismo tiempo una nueva lista enlazada que represente la suma.

```
Enter the first number:   538
Enter the second number: 4791
The sum is:              5329
```

La Estructura de Datos: Pila (Stack)

En la clase anterior vimos la estructura Lista Enlazada, en la que podemos colocar y remover un elemento en cualquier lugar de la lista, al inicio, en el medio o al final, sin embargo, hay situaciones y problemas para los cuales es necesario restringir la inserción y borrado de elementos, por ejemplo, permitiendo estas operaciones solo al inicio o al final de la lista, mas no en el medio. Dos estructuras son útiles en estas situaciones, las **pilas** y las **colas**.

Una **pila** es una estructura lineal que permite añadir o eliminar elementos solo en uno de sus extremos, al final de la lista, es decir en la *cima* (top) de la lista. Por lo tanto, si se desea retirar un elemento de la pila, será aquel que ingresó al último, es decir, el último que ingresa será el primero en salir (LIFO, last-in, first-out). En la siguiente figura se ilustra las dos operaciones básicas de las pilas.

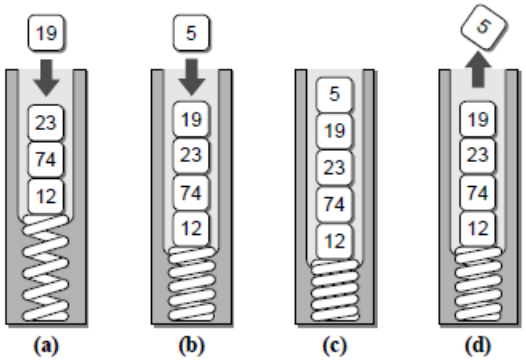


Figura 10.2.1: Vista abstracta de una pila: (a) colocando el valor 19; (b) colocando el valor 5; (c) pila resultante después de agregar 19 y 5; y (d) retirando el valor de la cima (5).

El diagrama UML de clases para las pilas es la siguiente, usando listas enlazadas.

Stack	
__top: Node **__size: int**	**Cabeza de la pila, objeto Node** **Tamaño de la lista**
__init__()	**Inicializa una pila vacía.**
isEmpty(): bool	**Retorna True si la pila está vacía.**
peek(): object	**Retorna el elemento que se encuentra en la cima de la pila sin eliminarlo de la pila.**
push(value:object): None	**Coloca un elemento en la cima de la pila.**
pop(): object	**Elimina el elemento de la cima de la pila y lo devuelve.**
getSize(): int	**Retorna el número de elementos de la pila.**

Implementación de Pilas:

Las pilas se pueden implementar de varias maneras, en Python se puede implementar usando las listas de Python y usando las listas enlazadas.

Implementación usando listas enlazadas

3. Implemente la clase `Stack` y todos sus métodos usando lista enlazada según el UML mostrado arriba. Para lo cual implemente la clase `Node` similar a la implementación que se realizó en `LinkedList` del laboratorio anterior. Escriba el programa de prueba.

```
class Node:
    def __init__(self, element):
        self.element = element
        self.next = None


class Stack :

   def __init__( self ):
        self.__top = None
        self.__size = 0

   def isEmpty( self ):

   def push(self, data):

   def pop( self ):

   def peek(self):

   def getSize(self):
```

4. Agregue a la clase `Stack` el método `__str__()` a fin de mostrar el contenido de la pila. Asimismo agregue el método `__len__()` a fin de usar la función `len()`. Escriba el programa de prueba.

5. Escriba un programa que invierta los elementos de una pila. Use la clase `Stack` implementada en el Ejercicio N° 4.
   a) Defina una función que reciba una pila y lo invierta usando **iteración.**
   b) Defina una función **`insert_at_bottom`** que recibe una pila y un valor como argumentos. La función adiciona el valor en la base de la pila (no en la cima) **usando recursión**.

      Defina una función **`reverse_stack`** que recibe una pila como argumento. La función invierte los elementos de la pila **usando recursión**. Sugerencia: la función **`reverse_stack`** invoca a **`insert_at_bottom`** a fin de invertir los elementos de la pila.

6. Agregue a la clase `Stack` el método `reverse()` que permita invertir los elementos de la pila.

OPCIONAL

7. Elabore una aplicación para comparar la performance de vuestra lista `LinkedList` con la lista de Python. Para lo cual puede crear una lista con 100,000 elementos usando el método **`insert()`** presente en ambas listas. Para ambas listas determine el tiempo de ejecución correspondiente a la creación de 100,000 elementos. Muestre los tiempos para ambas listas.

8. La notación postfix es una manera de escribir expresiones algebraicas sin usar paréntesis. Una expresión postfix se evalúa usando una pila.

   El algoritmo para evaluar una expresión postfix es el siguiente:
   a. Cree una pila para guardar los valores (operandos)
   b. Escanee la expresión dada y realice lo siguiente para cada elemento escaneado:
      b.1) si el elemento es un número, guárdelo en la pila (push)
      b.2) si el elemento es un operador, extraiga (pop) los dos operandos que están en la cima. Realice la operación con los valores extraídos y coloque el resultado en la pila.
   c. Cuando la expresión termine, el número en la pila es la respuesta final.

   Escriba un programa que solicite al usuario ingresar una expresión postfix y evalúe la expresión. Vuestra implementación de la pila debe usar listas enlazadas, no listas de Python.

   Ejemplo:
   Sea la expresión: **`"2 3 1 * + 9 -"`**. Debemos de escanear todos los elementos uno a uno.
   - Escanee "2", es un número, por lo que le enviamos a la pila. La pila contiene "2"
   - Escanee "3", nuevamente un número, envíelo a la pila, la pila ahora contiene "2 3" (de abajo arriba)
   - Escanee "1", nuevamente un número, envíelo a la pila, la pila contiene ahora " 2 3 1"
   - Escanee "*", es un operador, saque dos operandos de la pila, aplique el operador * sobre los operandos, es decir 3*1 que resulta 3. Colocamos el resultado "3" en la pila. La pila ahora contiene "2 3".
   - Escanee "+", es un operador, extraiga dos operandos de la pila, aplique el operador + sobre los operandos, es decir 2+3 que resulta 5. Colocamos el resultado "5" en la pila. Ahora la pila contiene "5".
   - Escanee "9", es un número, lo colocamos en la pila. La pila ahora contiene "5 9".
   - Escanee "-", es un operador, extraiga dos operandos de la pila, aplique el operador – sobre los operandos, es decir 5-9 que resulta -4. Colocamos el resultado en la pila. La pila ahora contiene "-4".
   - No hay más elementos a escanear, retornamos el elemento de la cima de la pila (que es el único elemento que quedó en la pila).

Guarde todos vuestros programas en una carpeta con el nombre su **Apellido** paterno seguido de vuestro **DNI**, luego comprima esta carpeta. Envíe este archivo a Katherine Navarro katherine.navarro@upch.pe especificando como asunto **Lab10.2**.